

Departamento de Estatística, Matemática Aplicada e Computação - IGCE
Bacharelado em Ciências da Computação

Amanda Vieira Reis
Edgar Auler Galvão de França
Filipi Peruca de Melo Pereira
Lucas Mateus Gonçalves de Góes
Nícolas Denadai Schmidt
Otávio Scarparo Souza

K-Means Clustering
Relatório de Implementação

Rio Claro
2025

SUMÁRIO

Introdução.....	3
K-Means.....	3
Pthreads.....	4
Pthreads - Estratégias.....	4
Pthreads - Struct de dados para as threads.....	4
Pthreads - Fase de Atribuição.....	5
Pthreads - Fase de Atualização.....	6
Pthreads - Compilação.....	7
OpenMP.....	7
OpenMP - Estratégias.....	7
OpenMP - Fase de Atribuição.....	7
OpenMP - Otimização do Cálculo de Distância.....	9
OpenMP - Fase de Atualização.....	9
OpenMP - Compilação.....	11
MPI.....	11
MPI - Estratégias.....	12
Distribuição Inicial dos Pontos.....	12
Distribuição dos Centróides (Iterativa).....	12
Fase de Atribuição (Cálculo Local).....	12
Fase de Atualização (Comunicação Coletiva e Redução).....	13
Acumulação Local.....	13
Redução Global.....	13
Recálculo e Distribuição.....	14
MPI - Compilação.....	14
Resultados.....	14
Análise.....	16

Introdução

O objetivo deste projeto é implementar e analisar versões paralelas do algoritmo K-Means utilizando três modelos de programação paralela distintos: Pthreads, MPI e OpenMP.

O projeto consiste em partir de uma implementação sequencial fornecida, compreender seu funcionamento, e então aplicar técnicas de paralelização adequadas a cada modelo para reduzir o tempo de execução ao processar grandes volumes de dados.

Além da implementação, o projeto envolve avaliar o desempenho obtido, comparar speedups e discutir vantagens e limitações de cada abordagem paralela.

K-Means

O algoritmo K-Means constitui um método não supervisionado de particionamento de dados amplamente utilizado em mineração de dados, reconhecimento de padrões e aprendizado de máquina. O método opera mediante processo iterativo que alterna entre duas fases principais:

- 1. Fase de Atribuição:** cada elemento do conjunto de dados é associado ao centróide mais próximo de acordo com alguma métrica (tipicamente, distância Euclidiana);
- 2. Fase de Atualização:** os centróides são recalculados como baricentro dos respectivos clusters (média aritmética das coordenadas dos elementos constituintes).

A complexidade computacional do algoritmo é $O(I \times M \times K \times D)$, sendo **I** o número de iterações, **M** a cardinalidade do conjunto de dados, **K** o número de partições desejadas e **D** a dimensionalidade do espaço vetorial. Dado o crescimento exponencial dos volumes de dados nas aplicações contemporâneas, a paralelização torna-se imperativa para viabilizar a execução do algoritmo em tempo computacionalmente aceitável.

Pthreads

Pthreads é uma API de baixo nível usada para criar paralelismo por meio de threads em sistemas Linux. Diferente de soluções mais simples, como OpenMP, ela coloca praticamente toda a responsabilidade nas mãos do programador: é preciso criar e encerrar threads manualmente, decidir como dividir as tarefas e cuidar de todos os mecanismos de sincronização para evitar erros como condições de corrida. Essa abordagem dá bastante liberdade e controle fino sobre o funcionamento do programa, permitindo otimizações detalhadas, mas também torna o desenvolvimento mais trabalhoso e sujeito a falhas.

Para isso, a API oferece ferramentas como mutexes, barreiras e variáveis de condição, que ajudam a coordenar o trabalho das threads e garantir que regiões críticas sejam acessadas com segurança. Em alguns algoritmos, Pthreads permite distribuir o processamento dos pontos entre várias threads, acelerando a etapa de classificação. Porém, cabe ao programador juntar manualmente os resultados parciais e sincronizar o acesso às estruturas compartilhadas. O desempenho pode ser excelente, mas a implementação acaba sendo mais extensa e complexa, o que reflete bem o seu lado “manual”.

Pthreads - Estratégias

Pthreads - Struct de dados para as threads

```
C/C++
typedef struct {
    Point *points;
    Point *centroids;
    int start_idx;
    int end_idx;
    int K;
    int D;
} ThreadData;
```

Pthreads - Fase de Atribuição

C/C++

```
void *assign_points_worker(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    for (int i = data->start_idx; i < data->end_idx; i++) {
        long long min_dist = LLONG_MAX;
        int best_cluster = -1;

        for (int j = 0; j < data->K; j++) {
            long long dist = euclidean_dist_sq(&data->points[i],
                                                &data->centroids[j], data->D);
            if (dist < min_dist) {
                min_dist = dist;
                best_cluster = j;
            }
        }
        data->points[i].cluster_id = best_cluster;
    }
    return NULL;
}

void assign_points_to_clusters(Point *points, Point *centroids, int M, int
K, int D, int num_threads) {
    pthread_t *threads = malloc(num_threads * sizeof(pthread_t));
    ThreadData *thread_data = malloc(num_threads * sizeof(ThreadData));

    int points_per_thread = M / num_threads;

    for (int i = 0; i < num_threads; i++) {
        thread_data[i].points = points;
        thread_data[i].centroids = centroids;
        thread_data[i].start_idx = i * points_per_thread;
        thread_data[i].end_idx = (i == num_threads - 1) ? M : (i + 1) *
points_per_thread;
        thread_data[i].K = K;
        thread_data[i].D = D;

        pthread_create(&threads[i], NULL, assign_points_worker,
&thread_data[i]);
    }

    for (int i = 0; i < num_threads; i++) {
        pthread_join(threads[i], NULL);
    }
    free(threads);
    free(thread_data);
}
```

Pthreads - Fase de Atualização

A decisão de não paralelizar a Fase de Atualização dos Centróides se baseia na Lei de Amdahl e no alto custo de sincronização. A Fase de Atribuição é o gargalo do K-Means, com complexidade dominante de $O(M \times K \times D)$, onde focamos nosso ganho de speedup. Paralelizar a Atualização, que é computacionalmente menos custosa, exigiria o uso de Mutexes (travas) ou Redução Manual para somar as contribuições de todas as threads de forma segura. Esse overhead de sincronização criaria contenda e serializaria o acesso às variáveis de soma, potencialmente anulando o pequeno ganho de desempenho e complexificando desnecessariamente o código.

```
C/C++
void update_centroids(Point *points, Point *centroids, int M, int K, int D)
{
    long long *cluster_sums = (long long *)calloc(K * D, sizeof(long long));
    int *cluster_counts = (int *)calloc(K, sizeof(int));

    for (int i = 0; i < M; i++) {
        int cluster_id = points[i].cluster_id;
        cluster_counts[cluster_id]++;
        for (int j = 0; j < D; j++) {
            cluster_sums[cluster_id * D + j] += points[i].coords[j];
        }
    }

    for (int i = 0; i < K; i++) {
        if (cluster_counts[i] > 0) {
            for (int j = 0; j < D; j++) {
                // Divisão inteira para manter os centroides em coordenadas
                // discretas
                centroids[i].coords[j] = cluster_sums[i * D + j] /
cluster_counts[i];
            }
        }
    }

    free(cluster_sums);
    free(cluster_counts);
}
```

Pthreads - Compilação

Em testes realizados, os melhores resultados foram obtidos com o uso das flags *-Ofast* e *-mavx2* do GCC.

OpenMP

OpenMP (Open Multi-Processing) é uma API de programação paralela baseada em memória compartilhada que utiliza o modelo de threads para paralelizar código C, C++ e Fortran de forma simples e portátil. Através de diretivas de compilação (`#pragma`), OpenMP permite ao programador adicionar paralelismo de maneira incremental ao código sequencial, com o compilador gerenciando automaticamente a criação, sincronização e destruição de threads.

A biblioteca oferece construções de alto nível como `parallel for`, `sections` e `tasks`, além de mecanismos de sincronização (`critical`, `atomic`, `barrier`) e controle de dados compartilhados (`shared`, `private`, `reduction`), facilitando a exploração de arquiteturas multi-core modernas sem a complexidade de gerenciar threads manualmente.

OpenMP é especialmente eficaz em aplicações com paralelismo de dados, onde loops independentes podem ser distribuídos entre threads, oferecendo boa escalabilidade com baixo esforço de implementação, embora seja limitado a sistemas com memória compartilhada e possa apresentar overhead de sincronização em casos com alta contenção de dados.

OpenMP - Estratégias

OpenMP - Fase de Atribuição

A fase de atribuição foi paralelizada mediante distribuição das iterações do loop externo entre as threads disponíveis, permitindo o processamento independente de cada ponto do conjunto de dados. Esta abordagem elimina dependências de dados entre iterações, uma vez que a atribuição de um ponto ao cluster mais próximo não

interfere no processamento de outros pontos, garantindo ausência de condições de corrida sem necessidade de mecanismos explícitos de sincronização.

Foram conduzidos experimentos sistemáticos com três políticas de escalonamento distintas com número de pontos reduzido para avaliar o impacto no desempenho paralelo. O schedule **static** apresentou speedup de 3.23x, distribuindo iterações em blocos contíguos de tamanho fixo entre threads. O schedule **guided** alcançou o melhor desempenho com speedup de 3.46x, utilizando blocos de tamanho decrescente para balanceamento dinâmico. O schedule **dynamic** obteve speedup de 2.64x, embora ofereça máxima flexibilidade, apresentou overhead elevado devido à atribuição individual de iterações.

A superioridade do schedule **guided** fundamenta-se em três aspectos principais. Primeiro, apresenta overhead de escalonamento intermediário, inferior ao **dynamic** que requer sincronização frequente para atribuição de iterações individuais. Segundo, oferece balanceamento de carga superior ao **static**, adaptando-se dinamicamente a variações no tempo de processamento entre iterações. Terceiro, demonstra particular eficácia em cenários onde as distâncias Euclidianas apresentam variabilidade significativa entre pontos, resultando em cargas computacionais heterogêneas. Esta heterogeneidade manifesta-se quando diferentes pontos requerem número variável de comparações até identificar o centroide mais próximo, situação na qual o escalonamento adaptativo do **guided** minimiza o tempo ocioso das threads.

```
C/C++
#pragma omp parallel for schedule(guided)
for (int i = 0; i < M; i++) {
    long long min_dist = LLONG_MAX;
    int best_cluster = -1;

    for (int j = 0; j < K; j++) {
        long long dist = euclidean_dist_sq(&points[i], &centroids[j], D);
        if (dist < min_dist) {
            min_dist = dist;
            best_cluster = j;
        }
    }
}
```

```

    }
}
points[i].cluster_id = best_cluster;
}

```

OpenMP - Otimização do Cálculo de Distância

O cálculo da distância Euclidiana constitui operação fundamental executada repetidamente durante a fase de atribuição, sendo invocada $M \times K$ durante cada iteração. Para otimizar, aplicamos a diretiva *SIMD* (*Single Instruction Multiple Data*), que instrui o compilador a vetorizar o loop de cálculo, explorando paralelismo em nível de instrução mediante unidades SIMD no processador.

```

C/C++
long long euclidean_dist_sq(Point *p1, Point *p2, int D) {
    long long dist = 0;
    #pragma omp simd reduction(+:dist)
    for (int i = 0; i < D; i++) {
        long long diff = (long long)p1->coords[i] -
p2->coords[i];
        dist += diff * diff;
    }
    return dist;
}

```

OpenMP - Fase de Atualização

A fase de atualização tem complexidade $O(M \times D + K \times D)$ e requer agregação de dados, apresentando maior desafio para a paralelização devido às dependências.

C/C++

```
#pragma omp parallel
{
    // Cada thread tem suas próprias estruturas locais
    long long *local_sums = (long long *)calloc(K * D,
sizeof(long long));
    int *local_counts = (int *)calloc(K, sizeof(int));

    // Fase 1: Acumulação local (sem contenção)
    #pragma omp for schedule(static)
    for (int i = 0; i < M; i++) {
        int cluster_id = points[i].cluster_id;
        local_counts[cluster_id]++;
        for (int j = 0; j < D; j++) {
            local_sums[cluster_id * D + j] +=
points[i].coords[j];
        }
    }

    // Fase 2: Redução global (com sincronização mínima)
    for (int i = 0; i < K; i++) {
        #pragma omp atomic
        cluster_counts[i] += local_counts[i];
        for (int j = 0; j < D; j++) {
            #pragma omp atomic
            cluster_sums[i * D + j] += local_sums[i * D + j];
        }
    }
    free(local_sums);
    free(local_counts);
}
```

A implementação utiliza armazenamento *thread-local* com estruturas privadas, eliminando a contenção de memória na acumulação. O escalonamento **static** maximiza a localidade espacial ao distribuir blocos contíguos. A sincronização, postergada para a redução global via operações atômicas, restringe-se a $K \times (D + 1)$ operações, contra as $M \times D$ da abordagem ingênua.

Os testes indicaram desempenho superior do **schedule static**, com *speedup* de 3.46x. O **schedule dynamic** alcançou 3.12x, enquanto o **guided** obteve 2.44x.

O **static** sobressai na fase de atualização por:

1. eliminar overhead de sincronização na distribuição;
2. maximizar a localidade espacial e reduzir *cache misses* via blocos contíguos;
3. permitir otimizações de compilador e *prefetching*. Ao contrário da fase de atribuição (carga heterogênea), a atualização possui carga uniforme, ideal para particionamento estático.

OpenMP - Compilação

Em testes realizados, os melhores resultados foram obtidos com o uso das flags *-Ofast* e *-mavx2* do GCC.

MPI

O MPI segue um caminho completamente diferente. Ele é um padrão mais comum para programação paralela e distribuída quando não há memória compartilhada entre os processos. Em vez disso, cada processo mantém seu próprio espaço de memória e se comunica com os demais enviando e recebendo mensagens. Isso faz com que MPI seja ideal para clusters e supercomputadores, onde escalabilidade e eficiência são cruciais.

A biblioteca inclui diversas rotinas de comunicação, tanto ponto a ponto quanto coletivas, como broadcast, scatter, gather e reduce, que ajudam a organizar o fluxo de dados entre os processos. MPI é usado para dividir o conjunto de pontos entre

diferentes processos, que fazem localmente os cálculos acumulando seus próprios resultados. Depois, esses dados são combinados por meio das operações coletivas. Apesar do custo adicional de comunicação e dos cuidados necessários para particionar bem os dados, MPI oferece ótimo desempenho em ambientes distribuídos e continua sendo uma das principais ferramentas da computação de alto desempenho.

MPI - Estratégias

A implementação paralela do K-Means com MPI adota o modelo de memória distribuída, sendo ideal para escalabilidade em clusters ou arquiteturas multi-core com processos independentes. A principal estratégia é a Decomposição de Dados, onde o conjunto total de **M** pontos é dividido estaticamente entre **P** processos.

Distribuição Inicial dos Pontos

O processo Rank 0 (o root) é responsável pela leitura inicial do conjunto completo de **M** pontos. Os dados de pontos são distribuídos para todos os processos usando a rotina `MPI_Scatterv`. Foi utilizada a versão `Scatterv` (variável) para lidar com a distribuição desigual de pontos quando **M** não é um múltiplo exato de **P**, garantindo que cada processo receba exatamente os seus dados. Essa distribuição é feita apenas uma vez, antes do laço principal de iterações, pois os pontos não mudam de posição durante o algoritmo.

Distribuição dos Centróides (Iterativa)

Os centróides mais recentes, recalculados pelo Rank 0 na iteração anterior, são comunicados a todos os processos usando a rotina `MPI_Bcast` (Broadcast) no início de cada iteração. Isso garante que todos os processos tenham acesso à posição atualizada de todos os **K** centróides antes de iniciar a Fase de Atribuição local.

Fase de Atribuição (Cálculo Local)

Esta fase é executada de forma independente por cada processo e é puramente computacional, sem comunicação interprocessos.

Cada processo calcula, para o seu subconjunto local de pontos (**M/P** pontos), a distância Euclidiana ao quadrado de cada ponto aos **K** centróides (recebidos via MPI_Bcast).

O cálculo de distância (euclidean_dist_sq) é otimizado com instruções SIMD AVX2 (com vetores de 256-bit), o que explora o paralelismo de dados ao nível de instrução dentro de cada processo MPI. O cluster_id de cada ponto é atualizado localmente, eliminando a necessidade de comunicação nesta fase, o que a torna altamente eficiente.

Fase de Atualização (Comunicação Coletiva e Redução)

Esta fase exige a agregação dos resultados parciais de todos os processos para calcular os novos centróides globais, sendo o principal ponto de contenção devido ao custo de comunicação:

Acumulação Local

Cada processo acumula localmente as somas das coordenadas (local_sums) e as contagens de pontos (local_counts) para os **K** clusters, utilizando apenas os pontos que lhe foram atribuídos. Esta acumulação é puramente sequencial e não requer sincronização local.

Redução Global

Os arrays locais (local_sums e local_counts) de todos os processos são somados e combinados no processo Rank 0 usando a rotina MPI_Reduce.

MPI_Reduce utiliza a operação MPI_SUM para somar os somatórios e contagens de todos os processos e depositar o resultado global (em global_sums e global_counts) no processo Rank 0.

C/C++

```
// Soma os somatórios de coordenadas de todos os processos no Rank 0.
MPI_Reduce(local_sums, global_sums, K * D, MPI_LONG_LONG,
MPI_SUM, 0,
           MPI_COMM_WORLD);
```

```
// Soma as contagens de pontos de todos os processos no Rank
0.
MPI_Reduce(local_counts, global_counts, K, MPI_INT, MPI_SUM,
0,
MPI_COMM_WORLD);
```

Recálculo e Distribuição

Apenas o Rank 0 recalcula os novos centróides (divisão da soma global pela contagem global). Estes novos centróides são distribuídos novamente no início da próxima iteração via MPI_Bcast, conforme descrito em 5.1.2.

A principal desvantagem desta abordagem reside no custo de comunicação das rotinas MPI_Bcast e MPI_Reduce, que se tornam o gargalo em sistemas com alta latência, limitando o speedup obtido.

MPI - Compilação

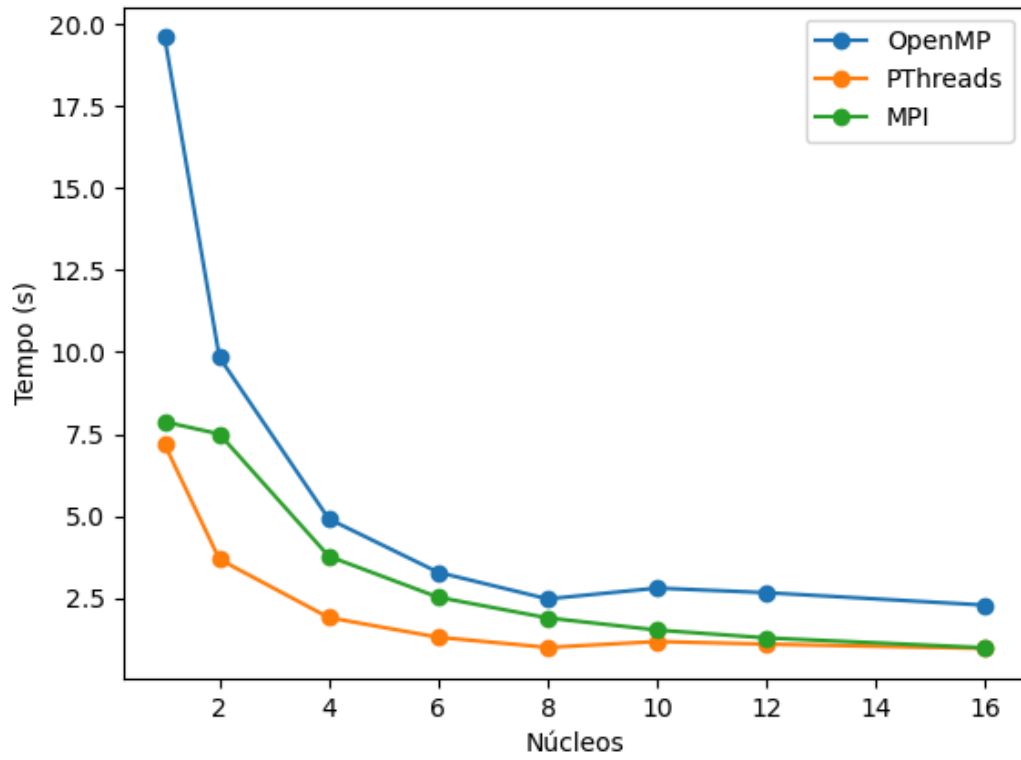
Em testes realizados, os melhores resultados foram obtidos com o uso do compilador wrapper mpicc e das flags de otimização -Ofast e -mavx2.

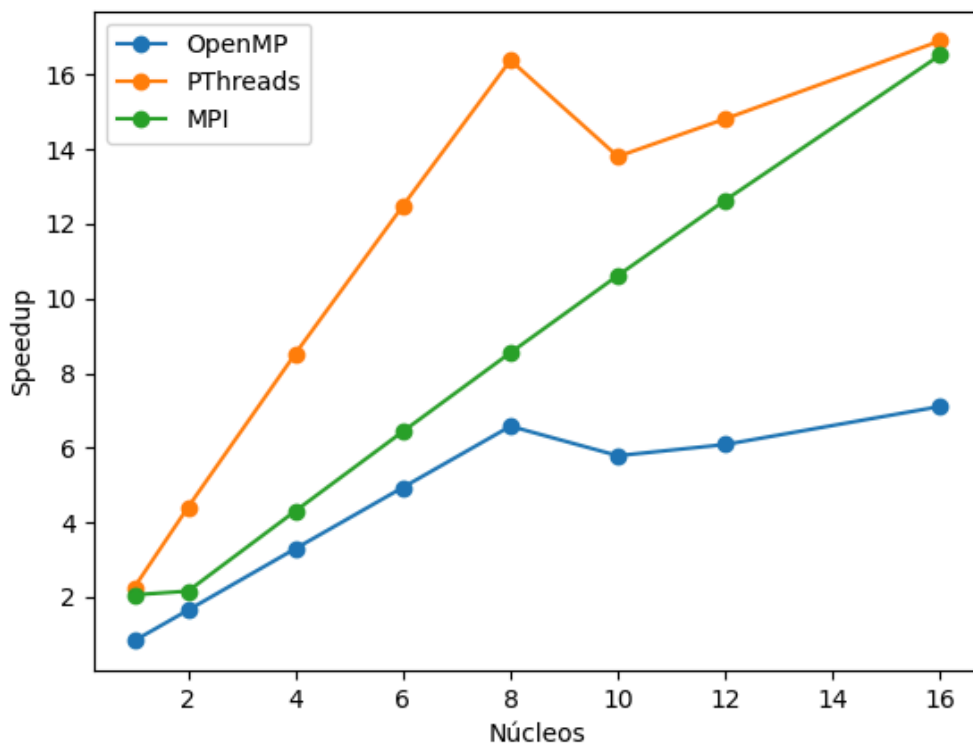
Resultados

Por fim, o avaliador foi executado em um computador com processador de 16 núcleos lógicos, com diferentes configurações de uso de núcleos:

núcleos	1	2	4	6	8	10	12	16
Sequencial	16.3s (1.00x)							
OpenMP	19.6s (0.83x)	9.84s (1.65x)	4.91s (3.30x)	3.28s (4.94x)	2.47s (6.58x)	2.80s (5.79x)	2.66s (6.09x)	2.28s (7.11x)
PThreads	7.18s (2.26x)	3.68s (4.41x)	1.90s (8.53x)	1.30s (12.49x)	0.99s (16.39x)	1.17s (13.81x)	1.09s (14.82x)	0.96s (16.91x)

MPI	7.87s (2.06x)	7.50s (2.16x)	3.77s (4.31x)	2.52s (6.44x)	1.89s (8.55x)	1.52s (10.62x)	1.28s (12.64x)	0.98s (16.53x)
------------	------------------	------------------	------------------	------------------	------------------	-------------------	-------------------	-------------------





Análise

Os melhores *speedups* foram obtidos com Pthreads, MPI e OpenMP, respectivamente. Para o Pthreads e OpenMP, o *speedup* foi proporcional até o uso de 8 núcleos e depois estagnou, enquanto para o MPI, o speedup foi proporcional até o uso de 16 núcleos.

Ao todo, a melhor configuração obtida (Pthreads, 16 núcleos) reduziu o tempo de execução sequencial de 16.3s para 0.96s, obtendo um *speedup* de cerca de 16.9 vezes.